

Trabajo práctico: búsqueda voraz y A* sobre un grafo dirigido

Curso: Inteligencia Artificial · **Semana y clase:** semana 3, clase 3 · **Modalidad:** individual **Grafo:** didáctico de 6 estados {S, A, B, C, D, G} · **Entregable:** notebook o script Python ejecutable

Cómo leer esta consigna: los recuadros como este son notas aclaratorias; no son parte de lo que se entrega, sino ayuda para entender el "por qué" de cada pedido.

Propósito

Implementar **tres estrategias de búsqueda** sobre el grafo dirigido de la clase —costo uniforme (UCS), voraz por el mejor primero y A— y *comparar qué camino devuelven, cuánto cuesta y cuántos estados expanden. El objetivo no es "hacer andar" un algoritmo: es distinguir rapidez de garantía. Sobre el mismo grafo, voraz y A pueden llegar al mismo resultado; la diferencia está en qué propiedad cada uno puede prometer.*

1. El grafo dirigido

Aristas dirigidas (no se puede recorrer una arista en sentido contrario) y costos:

Arista	Costo	Arista	Costo
S→A	2	S→B	2
A→C	2	A→D	5
B→D	2	C→G	3
D→G	6		

Heurística (estimación del costo restante hasta G):

Estado	S	A	B	C	D	G
h(n)	7	5	7	3	6	0

Regla de desempate: ante prioridades iguales, se extrae el que se insertó antes (orden de inserción, FIFO). Deben respetarla todos los algoritmos.

Nota: un grafo dirigido representa una relación asimétrica: S→A permite ir de S a A, pero no de A a S. El problema formal es $P=(S,A,T,s_0,G,c)$ con estados {S,A,B,C,D,G}, inicio S, objetivo {G}, transiciones las aristas listadas y costo c el valor de cada arista.

2. Estructura de datos y esquema común

Cada nodo conserva: (estado, padre, acción, g, h, f), donde g es el costo recorrido, h la estimación restante y f la prioridad. La implementación necesita además: una cola de prioridad, un diccionario `mejor_g[estado]` con el mejor costo conocido, descarte de entradas obsoletas y padres para reconstruir el camino. **El objetivo se comprueba al extraer, no al generar.**

Los tres algoritmos comparten el mismo esqueleto y solo cambian la prioridad:

```
insertar nodo inicial en frontera
mientras la frontera no esté vacía:
    extraer el nodo con menor prioridad (desempate por inserción)
    descartar si su g es obsoleto frente a mejor_g
    si es objetivo: devolver camino reconstruido
    relajar cada transición legal y actualizar mejor_g
devolver fracaso
```

Algoritmo	Prioridad	Qué privilegia
UCS	g	el camino más barato recorrido
Voraz	h	el estado que parece más cerca del objetivo
A*	g + h	el costo total estimado de la solución

3. Requerimientos de implementación

Código inicial opcional

Para comenzar con estructuras de Python básicas, se puede representar cada nodo mediante un diccionario. El campo `padre` guarda una referencia al nodo anterior y permite reconstruir el camino al final:

```
def crear_nodo(estado, padre=None, accion=None, g=0, h=0):
    return {
        "estado": estado,
        "padre": padre,
        "accion": accion,
        "g": g,
        "h": h,
        "f": g + h,
    }

raiz = crear_nodo("S", g=0, h=7)
nodo_a = crear_nodo(
    "A",
    padre=raiz,
    accion="S -> A",
    g=2,
    h=5,
)

frontera = [raiz, nodo_a]
nodo = min(frontera, key=lambda n: n["f"])
frontera.remove(nodo)
```

Este código solo muestra la representación inicial y la extracción manual del nodo con menor `f`; todavía no implementa A completo. `nodo["g"]` es el costo acumulado del camino hasta ese nodo, mientras que `nodo["f"]` es su prioridad para A. Para UCS, voraz y A* se deberá cambiar la clave de prioridad a `g`, `h` o `g+h`, respectivamente. Luego se puede reemplazar la lista por `heapq`, siempre que se conserve el desempate estable por orden de inserción.

- Usar Python con una cola de prioridad (p. ej. `heapq`) y desempate estable por orden de inserción.
- No devolver el camino "a mano" ni hardcodear el resultado: el camino debe reconstruirse desde los padres.
- Para cada algoritmo devolver: camino, costo, estados **generados**, estados **expandidos**, frontera máxima y reaperturas (veces que mejora `mejor_g` de un estado ya conocido).
- Registrar una **traza**: por cada expansión, el estado extraído y el contenido de la frontera (con su prioridad).

4. Pseudocódigos de referencia

Los tres algoritmos comparten el control de repetidos, la reconstrucción del camino y la prueba del objetivo al extraer. Cambia la prioridad de la frontera:

```
COSTO-UNIFORME(problema):
  frontera <- prioridad por g
  mejor_g[inicial] <- 0; insertar raíz con g=0
  mientras frontera no esté vacía:
    nodo <- EXTRAER-MINIMO(frontera)
    si nodo es obsoleto: continuar
    si es objetivo: devolver camino
    relajar sucesores e insertar mejoras por g
  devolver fracaso
```

```
VORAZ(problema, h):
  frontera <- prioridad por h
  mejor_g[inicial] <- 0; insertar raíz con h=h(inicial)
  mientras frontera no esté vacía:
    nodo <- EXTRAER-MINIMO(frontera)
    si es objetivo: devolver camino
    para cada sucesor: si mejora mejor_g, insertar con h(sucesor)
  devolver fracaso
```

```
A-ESTRELLA(problema, h):
  frontera <- prioridad por f=g+h
  mejor_g[inicial] <- 0; insertar raíz con f=h(inicial)
  mientras frontera no esté vacía:
    nodo <- EXTRAER-MINIMO(frontera)
    si nodo es obsoleto: continuar
    si es objetivo: devolver camino
    para cada sucesor: si mejora mejor_g, insertar con nuevo_g+h
  devolver fracaso
```

En los tres casos, **relajar** significa calcular el nuevo costo, comparar con **mejor_g**, registrar el padre e insertar solo si aparece una mejora. La implementación debe conservar el desempate por orden de inserción.

5. Verificación y comparación

1. Reproducir con tu código la traza completa de UCS, voraz y A* sobre el grafo.
2. Completar la tabla comparativa:

Resultado	UCS	Voraz	A*
Camino			
Costo			
Prioridad	g	h	g+h
Expandidos antes de extraer G			

6. Preguntas de análisis

1. ¿Por qué voraz y A* coinciden en este grafo? ¿Qué condición del grafo y de la heurística lo explica?
2. ¿Garantiza voraz devolver el camino de menor costo en general? Justifícalo con la propiedad de su prioridad (no con este ejemplo).
3. ¿Qué ocurre si se usa $h = 0$ en A*? ¿Con qué algoritmo coincide entonces?
4. ¿Hubo reaperturas en UCS? ¿Y en A* y voraz? ¿Por qué?
5. ¿"Expandir menos estados" significa "camino más barato"? Relacionalo con lo que muestran UCS y voraz aquí.

7. Producto esperado

Entregar una copia ejecutable que corra de punta a punta, con: identificación del estudiante; representación del grafo y la heurística; los tres algoritmos; la traza de cada uno; la tabla comparativa completa; y las respuestas a las cinco preguntas de análisis.

No se evalúa llegar a un resultado predeterminado. Se evalúa que la implementación sea reproducible, que la regla de desempate y la prueba del objetivo estén bien aplicadas, y que las afirmaciones sobre garantías sean las correctas.